

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-03 (0fb5559+수정중) → 2026-08-10 · 801d39a+수정중 | 1개 파일 · +38 / -9 줄 · 코드 6쪽 | 새 코드북 70쪽

바뀐 내용

- 버스트 탐지가 사람 눈보다 둔하던 것을 고쳤습니다. 스펙트로그램에 보이는 약한(6dB급) 버스트와 짧은(200샘플급) 버스트를 이제 잡고, 잡음 간격이 또렷한 버스트들은 붙이지 않고 따로 셉니다.
- 잡음뿐인 파일, 신호가 짝 찬 파일은 예전처럼 파일 전체를 한 구간으로 돌려줍니다 — 합성 1680조합에서 오경보 0 그대로입니다.
- 버스트 안에 임펄스가 끼면 엉뚱한 버스트를 고르던 것도 이제 안전하게 전체 구간으로 물러납니다. 새로 칠 파일은 dsp.py 하나입니다.

□ `signus/dsp.py`

+38

-9

2쪽

코드북 15쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/dsp.py +38 -9 · 코드북 15쪽 · 새로 쓸 줄 88-102, 114-121, 124-127, 136, 142, 145-153

```

1 1 """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2 2 anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3 3
4 4 from fractions import Fraction
5 5
6 6 import numpy as np
7 7 from scipy.ndimage import uniform_filter1d
8 8 from scipy.signal import hilbert, resample_poly, welch
9 9
10 10 from ._accel import dd_carrier
11 11 from .constellations import ideal_points
12 12
13 13
14 14 def analytic(x: np.ndarray) -> np.ndarray:
15 15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16 16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17 17     overflows on the  $|x|^2$  the estimators take)."""
18 18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19 19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows  $|x|^2$  is corrupt t
20 20     oo
21 21     if np.iscomplexobj(x):
22 22         return x.astype(np.complex128)
23 23     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
24 24
25 25 def _parab(ydb: np.ndarray, k: int) -> float:
26 26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27 27     if k <= 0 or k >= ydb.size - 1:
28 28         return 0.0
29 29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30 30     d = a - 2 * b + c
31 31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32 32     # (d~0) or non-max triple can explode the ratio and drive baud negative -> a resample crash
33 33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34 34
35 35
36 36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37 37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38 38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39 39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40 40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41 41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42 42     lab = np.zeros(a.size, dtype=int)
43 43     for _ in range(iters):
44 44         d = np.abs(a - c[0])
45 45         lab = np.zeros(a.size, dtype=int)
46 46         for j in range(1, k):
47 47             dj = np.abs(a - c[j])
48 48             closer = dj < d
49 49             lab[closer] = j
50 50             d[closer] = dj[closer]
51 51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52 52         if np.allclose(nc, c):
53 53             break
54 54         c = nc
55 55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56 56
57 57

```

```

58 58 # --- burst detection -----
59 59
60 60 _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, no
   61 t a
61 61 # burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/Q
   62 AM
62 62 # peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared o
   63 ver
63 63 # a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with marg
   64 in.
64 64
65 65
66 66 def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
67 67     """Dual-threshold energy detector with an Otsu log-power floor; return every
68 68     merged burst in time order, or [(0, size)] when nothing stands out."""
69 69     n = x.size
70 70     win = max(64, n // 1000)
71 71     pw = np.abs(x) ** 2
72 72     ps = uniform_filter1d(pw, win)
73 73     lp = np.log10(ps + 1e-20)
74 74
75 75     # Otsu split of the log-power histogram: a floor that survives an 80%-full
76 76     # record because it is drawn from the (small) below-threshold noise class.
77 77     hist, edges = np.histogram(lp, bins=128)
78 78     centers = (edges[:-1] + edges[1:]) / 2
79 79     w = np.cumsum(hist).astype(float)
80 80     m = np.cumsum(hist * centers)
81 81     mb = m / np.where(w > 0, w, 1)
82 82     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
83 83     btw = w * (w[-1] - w) * (mb - mf) ** 2
84 84     thr = centers[int(np.argmax(btw))]
85 85     noise = lp[lp < thr]
86 86     floor = noise.mean() if noise.size else lp.min()
87 87
88 + # The Otsu floor is only a NOISE floor when the histogram really is bimodal. On a record
89 + # with no noise stretch (a signal filling the capture, or pure noise) the split lands
90 + # inside one mode and "floor" is just that mode's lower half -- the lowered bars below
91 + # would then promote envelope jitter to micro-bursts (a 60k-sample 16qam multipath record
92 + # yielded a 209-sample fragment). Class separation < 0.4 decades = no distinct floor.
93 + sig = lp[lp >= thr]
94 + if not sig.size or float(sig.mean() - floor) < 0.4:
95 +     return [(0, n)]
96 +
97 + # Thresholds are on WIN-SMOOTHED log power, where the noise spread is ~0.054/sqrt(win/64)
98 + # decades: +5 dB is already >9 sigma at the smallest window, so lowering the old +10/+6 dB
99 + # bars costs no noise false alarms while making ~6 dB bursts (clearly visible on a
100 + # spectrogram, invisible to the old bar) detectable.
101 + above_hi = lp >= floor + 0.5 # 5 dB over floor
102 + above_lo = lp >= floor + 0.3 # 3 dB
90 103 dif = np.diff(above_lo.astype(np.int8))
91 104 starts = list(np.where(dif == 1)[0] + 1)
92 105 ends = list(np.where(dif == -1)[0] + 1)
93 106 if above_lo[0]:
94 107     starts.insert(0, 0)
95 108 if above_lo[-1]:
96 109     ends.append(n)
97 110 runs = [(s, e) for s, e in zip(starts, ends, strict=True) if above_hi[s:e].any()]
98 111 if not runs:
99 112     return [(0, n)]
100 113

```

```

114 + # BOTH gates are tied to the smoothing width, NOT the record length. Record-scaled gates
115 + # (n//200) dropped a field capture's strong 200-sample packet train wholesale (65k record,
116 + # mlen 325 -> [(0,n)] fallback) and merged bursts across visible noise gaps. The smoother
117 + # widens a true burst of L samples into a ~L+win run and smears an impulse into a ~win run,
118 + # so mlen = win + 128 passes any real burst >=~128 samples at EVERY record length while
119 + # still length-dropping impulse smears (the _SPIKE_PAR shape test below stays as backstop).
120 + gap = max(64, win) # a visible noise gap (> ~2*win true) = separate bursts
121 + mlen = win + 128
106 122 merged = [list(runs[0])]
107 123 for s, e in runs[1:]:
124 + # A real inter-burst gap returns to the noise floor; a marginal-level burst (between
125 + # the lo and hi bars) dips just BELOW lo without ever reaching the floor and would
126 + # otherwise shatter into fragments. Merge across any valley that stays off the floor.
127 + if s - merged[-1][1] < gap or float(lp[merged[-1][1]:s].mean()) > floor + 0.15:
109 128     merged[-1][1] = e
110 129     else:
111 130         merged.append([s, e])
112 131 # The mlen cap alone lets a short high-energy transient survive in a long record: the smooth
113 132 er
114 133 # spreads a single impulse into a ~win-wide run that clears 1024, and its energy can outrank
115 134 # the real signal so analyze() auto-selects the blip. A modulated burst has a near-flat
116 135 # envelope (raw peak/mean power ~ a few); an impulse smear is one spike over noise (ratio ~w
117 136 in).
118 137 # Reject runs whose RAW power is spike-dominated -- this separates them by shape, at any len
119 138 gth.
136 + out, spiked = [], 0
118 137 for s, e in merged:
119 138     if e - s < mlen:
120 139         continue
121 140     seg = pw[s:e]
122 141     if float(seg.max()) / (seg.mean() + 1e-30) > _SPIKE_PAR:
142 +         spiked += e - s
123 143         continue
124 144     out.append((int(s), int(e)))
145 + # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
146 + # inside the strongest burst killed its run, returning the survivors would hand analyze()
147 + # the wrong emitter -- fall back instead. Only spike kills count: mlen drops are the
148 + # routine blip filter (borderline-short runs at a large win), and lo-only energy (weak
149 + # siblings, a second signal between the bars) never qualified -- counting either of those
150 + # threw away clean partial detections.
151 + covered = sum(e - s for s, e in out)
152 + if out and covered < 0.6 * (covered + spiked):
153 +     return [(0, n)]
125 154     return out or [(0, n)]
126 155
127 156
128 157 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
129 158     """The most energetic burst of find_bursts."""
130 159     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
131 160
132 161
133 162 # --- carrier estimation -----
134 163
135 164 def est_carrier(
136 165     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
137 166     nfft: int = 65536, blocks: int = 4,
138 167 ) -> tuple[float, int, bool]:
139 168     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
140 169     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
141 170     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))

```

```

142 171 best_par, best_s, sym = -1.0, None, powers[0]
143 172 for p in powers:
144 173     xp = x ** p
145 174     blk = xp.size // blocks
146 175     segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
147 176     acc = np.zeros(nfft)
148 177     for s in segs:
149 178         acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
150 179     spec = acc / len(segs)
151 180     par = spec.max() / spec.mean()
152 181     if par > best_par:
153 182         best_par, best_s, sym = par, spec, p
154 183
155 184     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
156 185         return 0.0, int(powers[0]), True
157 186 ydb = 20 * np.log10(best_s + 1e-20)
158 187 k = int(np.argmax(best_s))
159 188 f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
160 189 fc = f_peak / sym
161 190 ambiguous = abs(sym * fc) > 0.4 * fs
162 191 return float(fc), int(sym), bool(ambiguous)
163 192
164 193
165 194 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
166 195     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
167 196     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
168 197     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
169 198     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
170 199     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
171 200     p = np.maximum(pxx - np.median(pxx), 0)
172 201     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
173 202     # 8psk beyond 0.3125*fs with no candidate).
174 203     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
175 204     if abs(fc + k * fs / sym) < 0.5 * fs]
176 205     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
177 206     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum
178 207     ()))
179 208
180 209 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
181 210     """Downconvert by fc (complex heterodyne)."""
182 211     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
183 212
184 213
185 214 # --- symbol rate + rolloff -----
186 215
187 216 def est_baud(
188 217     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
189 218     blocks: int = 4,
190 219 ) -> tuple[float, float]:
191 220     """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
192 221     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
193 222     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
194 223     u = np.abs(x) ** 2
195 224     u = u - u.mean()
196 225     blk = u.size // blocks
197 226     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
198 227     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
199 228     acc = np.zeros(nfft // 2 + 1)
200 229     for s in segs:

```

```

201 230         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
202 231     spec = acc / len(segs)
203 232     freqs = np.fft.rfftfreq(nfft, 1 / fs)
204 233
205 234     lo = lo if lo is not None else 0.005 * fs
206 235     hi = hi if hi is not None else 0.45 * fs
207 236     band = (freqs >= lo) & (freqs <= hi)
208 237     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
209 238         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
210 239     idx = np.where(band)[0]
211 240     k = idx[int(np.argmax(spec[idx]))]
212 241     ydb = 20 * np.log10(spec + 1e-20)
213 242     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
214 243     conf = spec[k] / (np.median(spec[band]) + 1e-20)
215 244     return float(baud), float(conf)
216 245
217 246
218 247 def occupied_bw(x: np.ndarray, fs: float) -> float:
219 248     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way
220 249     from the 25th-percentile floor to the 90th-percentile in-band level, walk
221 250     outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
222 251     Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
223 252     f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
224 253     floor = np.percentile(pxx, 25)
225 254     order = np.argsort(np.abs(f))
226 255     idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
227 256     if idx.size == 0:
228 257         return 0.0
229 258     last = idx[-1]
230 259     return float(2 * np.abs(f[order][last]))
231 260
232 261
233 262 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
234 263     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
235 264     n = x.size
236 265     nper = min(8192, max(256, n // 8))
237 266     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
238 267     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
239 268     af = np.abs(f)
240 269     inband = np.median(pxx[af < 0.25 * baud])
241 270     noise = np.median(pxx[af > 0.9 * baud])
242 271     thr = noise + 0.10 * (inband - noise)
243 272     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
244 273     if not band.any():
245 274         return 0.35
246 275     raw = 2 * af[band].max() / baud - 1
247 276     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
248 277
249 278
250 279 # --- resampling + matched filter -----
251 280
252 281 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
253 282     """Rational resample to exactly sps samples/symbol."""
254 283     r = Fraction(sps * baud / fs).limit_denominator(2000)
255 284     return resample_poly(x, r.numerator, r.denominator)
256 285
257 286
258 287 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
259 288     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
260 289     |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""

```

```

261 290     m = (span * sps) // 2
262 291     t = np.arange(-m, m + 1) / sps
263 292     a = alpha
264 293     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
265 294     den = np.pi * t * (1 - (4 * a * t) ** 2)
266 295     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
267 296     h[t == 0] = 1 - a + 4 * a / np.pi
268 297     if a > 0:
269 298         s = np.isclose(np.abs(t), 1 / (4 * a))
270 299         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
271 300             + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
272 301     return h / np.sqrt(np.sum(h ** 2))
273 302
274 303
275 304 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
276 305     """Apply the RX matched RRC filter."""
277 306     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
278 307
279 308
280 309 # --- timing recovery -----
281 310
282 311 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
283 312     """Block-wise 0erder-Meyr timing recovery; returns `out` samples/symbol
284 313     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
285 314     Per block the spectral-line phase gives the offset; unwrapped across blocks
286 315     to track clock drift, then interpolated per symbol."""
287 316     n = sps
288 317     nsym = x.size // n
289 318     if nsym < 1:
290 319         return x[:0].astype(np.complex128)
291 320     xt = x[:nsym * n]
292 321     p = np.abs(xt) ** 2
293 322     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
294 323     bs = block * n
295 324     if xt.size <= bs: # single block: one global fractional offset
296 325         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
297 326     else:
298 327         nblk = xt.size // bs
299 328         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
300 329         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
301 330         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
302 331         pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
303 332         idx = np.arange(xt.size)
304 333         return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
305 334
306 335
307 336 # --- decision-directed carrier sync -----
308 337
309 338 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarra
310 339     y:
311 340     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
312 341     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
313 342     phase update depends on the prior decision) -> numba kernel when available."""
314 343     pts = ideal_points(mod)
315 344     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
316 345     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
        np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```